

Lecture 11 Hashing

Searching

Determine whether a particular record is in the collection or not. (may be sorted or not; may be empty or not)

Generally:

- **Linear** Search if the array is **not sorted**. Average and worst case: $O(n)$
- **Binary** Search if the array is **sorted**. Average and worst case: $O(\log n)$

Key Fields

Records can hold **complex constructs** which have many fields of information in each record. The field to organize the records is called the **key field**.

Binary Search

Input:

- A sorted array and its size
- A value to be searched for

Output:

- If found, return the corresponding index
- If not found, return error value

The method is recursive:

- Compare the value with the middle value of the array
- If equal, return mid
- If less, find in the left half; if larger, find in the right half
- If left bound encounter right bound, return error

Hashing

Hashing table can reach $O(1)$.

Hash function is a function that input a value of the key to search for and output where the value is.

Hashing Table

The simplest kind of hash table is an array of some fixed size containing records. Each record has its key and for each key we define the **hash function**:

$$f(x) = \text{position of } x \text{ in } ht[]$$

A **collision** occurs when two nonidentical keys output the same value. i.e.

$$f(x_1) = f(x_2) \text{ when } x_1 \neq x_2.$$

Remaining Problems

- Choose a hash function
- What to do when collisions happen
- Decide the size of table

Hash Function

- Must **easy(fast)** to compute
- Should distribute the keys into table **evenly**. i.e.

$$\text{Probability}(f(x) = i) = 1/\text{Tablesize}$$

Such a hash function is called a **uniform hash function**.

For integer keys:

$$f(x) = x \% \text{Tablesize}$$

Normally, we choose **prime number** as table size to avoid hash collision.

For string keys:

$$f(x) = \left(\sum x[i] \right) \% \text{Tablesize}$$

where $x[i]$ is the code of the i -th symbol. But for large table size, there exist a **large amount of waste space**.

Other choice:

$$f(x) = \left(\sum x[N - i - 1] * 32^i \right) \% \text{TableSize}$$

to utilize more table space. Choose 32 rather 27 to directly shift.

Solving Collisions

- Separate Chaining
- Open Addressing

- Linear probing
- Quadratic probing
- Double hashing

Separate Chaining

Keep a **linked list** of all keys that hash to the same value.

Loading density $\lambda = n/\text{Tablesize}$ where n is the total number of keys in the hash table.

Open Addressing

Find another empty cell to solve collision.

$$hi(X) = (\text{Hash}(X) + f(i)) \bmod \text{Tablesize}$$

where $f(i)$ is called **collision resolving function** and $f(0) = 0$.

Linear Probing

$$f(i) = i$$

i.e., if occupied, find the next position.

But it may cause **primary cluster**. i.e., keys that hash to **neighbor positions will contribute a larging cluster**.

Quadratic Probing

$$f(i) = i^2$$

It can be computed by $f(i) = f(i - 1) + 2i - 1$

However, it causes **second clustering**. i.e., keys that hash to the some position will **probe the same alternative**.

Generally, for open addressing hash table, choose $\lambda < 0.5$.

When delete an element in the hashing table, it will cause error (return not found) if directly delete. So we use **lazy delete**. i.e., **set its status into deleted but not delete the record in the bucket**. When searching, if find deleted status, searching the next by collision resolving function. When inserting, we can **reuse the deleted position**.

Double Hashing

$$f(i) = i * \text{hash}_2(x)$$

i.e., use another hash function. It resolve primary clustering and second clustering.
But we should ensure

- $hash_2(x) \neq 0$
- all cells can be probed.

Generally,

$$hash_2(x) = R - (x \% R)$$

where R is a prime smaller than table size.

Rehashing

When the loading density is close to 0.5, we need to

- build another table that is about twice as large
- Scan down the entire original hash table for non-deleted elements
- Use a new functions to hash those elements into the new table.